我的面试题 · 技术问答

112 道技术面试题 · 14 个主题 · 答题模板

rag系统

1. 如何保证意图识别的稳定性?

意图识别要尽量做成「规则优先 + 模型补充 + 置信度决策」，而不是完全依赖 LLM 自由判断。

第一，定义清晰的意图 Schema：意图名称、典型说法、必填槽位、反例，避免边界模糊。能做规则匹配的先走规则，比如明确的命令、菜单、业务关键词。

第二，模型只处理规则覆盖不了的模糊输入，并输出结构化结果和置信度。温度调低，示例覆盖同义表达和多意图情况。

第三，加置信度门控：高置信度直接路由；中间区间给候选让用户确认；低置信度追问澄清，不要硬路由。兜底路径要存在，比如统一走通用 RAG 或转人工。

第四，用评测集持续验证：同义改写、错别字、多意图、否定表达都要覆盖。上线后记录每次路由决策和用户纠正，定期回流。

最后，意图识别不是一次分类，多轮对话里要结合上下文和已有槽位，避免用户补充信息后仍按第一轮意图走。

2. 多路由结果怎么合并给LLM?

先统一表示，再融合排序，最后按预算组装，不能把几路结果原样拼接。

第一步，每路结果统一成同一种结构：来源、内容、分数、元数据、时间、权限。不同来源的分数不能直接比较，先做归一化或只用排名。

第二步，去重和融合。按文档 ID、内容哈希或实体去重；融合可以用 RRF（倒数排名融合）或加权投票，混合检索场景 RRF 更稳。

第三步，过滤和重排。按租户、权限、时效过滤，再用 Rerank 统一排序；不同来源可以保留少量配额，避免某一路完全被挤掉。

第四步，组装上下文。按相关性和多样性取 Top-K，给每段标来源、时间和置信度；总 Token 超预算时先截断低分内容，而不是平均压缩。

最后，把路由决策和融合结果记录到执行轨迹里，方便回答「为什么用了这段内容」。

3. BadCase如何回流到BenchMark?

我会把 BadCase 回流做成一条固定流水线，而不是人工随手加几条。

第一，采集：记录 query、意图、路由、召回文档、rerank 分数、上下文、最终答案、用户反馈和执行轨迹。

第二，归因：人工或自动判断失败环节，是 query 理解、召回、排序、上下文组装还是生成；标出正确文档或正确做法。

第三，清洗：脱敏、去掉与业务无关的偶发因素，补上期望输出和评分标准。

第四，入库：按场景、难度、失败类型分组，版本化管理；高频繁、高风险的问题优先。

第五，回归：进入评测集后，每次改动都跑；修复后确认从失败变通过，并防止其他场景退化。

要注意防止数据泄漏：评测样本不要出现在 Prompt 示例或训练数据里，否则分数会虚高。

4. chunk如何根据BadCase优化？（chunk大小，chunk策略）

先判断 BadCase 属于哪种失败，再调 chunk，而不是盲目改大小。

如果是「答案被切断了」：说明 chunk 太小或边界不好，改成按标题、段落、语义切分，增加适量重叠，或者用「小块检索、大块生成」的父子结构。

如果是「召回了但噪声太多」：说明 chunk 太大，尝试缩小 chunk，或按句子/要点切分，并在检索后做 rerank。

如果是「跨段落信息没拼起来」：检查依赖关系，增加父子块、相邻块扩展或摘要索引。

如果是「元数据缺失导致过滤错」：补文档结构、章节路径、时间和权限字段。

调整后必须用评测集 A/B 验证：看 Recall@K、精确率、答案正确率和延迟，不要只看单个 case。chunk 策略没有通用最优解，要靠数据和业务文档类型决定。

5. query改写后，召回效果差怎么办?

先确认是改写本身错了，还是召回链路的问题。

第一，做对照实验：原始 query、改写 query、多路 query 分别召回，比较正确文档的排名。如果原始 query 更好，说明改写在丢信息或引入偏差。

第二，保留原始 query 做多路召回，融合结果，而不是只信改写版本。改写置信度低时直接回退原文。

第三，约束改写目标：只做指代消解、术语归一、拼写纠正，不要让模型自由扩写；HyDE 这类方法要谨慎，容易把假设当成事实。

第四，检查改写后的 query 是否超出索引语言/领域，或者过长导致语义稀释。

第五，把失败的改写样本回流到评测集，优化 Prompt 和示例。改写不是越多越好，能提升召回且不改变原意才有价值。

6. query召回率高，但精确率低怎么办?

召回率高说明正确文档大概率在里面，问题在排序和噪声控制。

第一，加 Rerank，对候选做 query-doc 精细打分，这是提升精确率最直接的手段。

第二，缩小 Top-K 或按分数阈值截断，不要把所有召回都塞给模型。

第三，用 metadata 过滤：时间、类型、部门、权限、文档版本，先排除不相关范围。

第四，优化 chunk：太大导致一段里混入多个主题，可以拆小；太小导致语义不完整，可以用父子块。

第五，做多样性控制，比如 MMR，避免返回一堆内容重复的片段。

第六，检查 query 是否有歧义，必要时澄清或改写。

最后用 Precision@K、NDCG 和答案正确率一起评估，不能只看召回率。

7. chunk的metadata保存什么?

至少保存四类信息。

第一，定位信息：doc_id、chunk_id、parent_id、标题、章节路径、页码或字符偏移，用于引用和回溯。

第二，业务信息：租户、部门、密级、权限范围、文档类型、语言、业务标签。

第三，版本与时间：文档版本、chunk 版本、创建/更新时间、有效期，用于冲突处理和失效。

第四，技术信息：内容哈希、embedding 模型及版本、分块策略版本、来源 URL。

如果做混合检索，还可以存关键词、实体、摘要，方便 BM25 和过滤。

metadata 不是越多越好，但缺少关键字段会导致权限、更新和引用都做不好。设计时按「检索过滤、引用、失效、审计」四个用途倒推。

8. 召回上下文冲突怎么办?

不要静默选一个，也不要把冲突内容直接丢给模型让它自由发挥。

第一，检测冲突：同一实体/字段出现不同值，或者新旧版本文档说法不一致。

第二，按可信度排序：来源权威性、文档版本、更新时间、租户/部门范围。通常情况下，新版本、权威来源优先。

第三，在上下文里显式标注冲突：告诉模型「A 文档说 X，B 文档说 Y，时间分别是……」，让它基于证据回答，或者说明存在冲突。

第四，高风险场景不要自动裁决，转人工确认或要求用户补充条件。

第五，从数据侧治理：建立版本链和失效机制，避免旧文档继续被召回；对冲突高频的领域做专门的主数据管理。

冲突处理的目标是「可解释」，不是「看起来只有一个答案」。

9. 数据入库冲突怎么办?

核心是幂等写入 + 版本管理 + 冲突隔离。

第一，用稳定主键做 upsert：doc_id + chunk_id（或内容哈希），重复写入不产生重复数据。

第二，保留版本链：同一文档的新版本写入后，旧版本标记失效，而不是直接覆盖；需要能回滚和追溯。

第三，检测内容冲突：同一主键但内容差异很大，或者不同来源对同一事实说法不一致，进入隔离区，人工确认后再合并。

第四，写入要可审计：谁、什么时候、从哪个来源写入，方便定位问题。

第五，删除用 tombstone（软删除）+ 异步清理，避免删除过程中被重新召回。

第六，批量导入要做去重和限流，避免重复任务把索引打爆。

入库冲突本质上是数据治理问题，不能只靠向量库的 upsert 解决。

10. query查询与生成的时候，知识库发生了变更怎么办?

关键是让一次查询看到「一致的快照」，而不是边查边变。

第一，索引版本化：每次全量/增量更新生成一个新版本，查询开始时固定版本，后续检索都用这个版本。

第二，如果做不到快照，至少记录本次查询命中的文档版本，生成时以已召回的内容为准，并在引用里标注版本。

第三，变更窗口内做提示：如果检测到查询期间索引发生重大变更，可以重新检索一次，或者告知用户结果基于旧版本。

第四，缓存要按索引版本失效，避免返回旧答案。

第五，对高一致性要求的场景，可以用读写分离 + 版本号校验，或者短暂排队等索引追平。

大多数 RAG 场景可以接受「最终一致」，但必须让用户知道答案基于哪个版本，尤其是制度、价格、配置这类会变化的知识。

11. rag系统中文档频繁新增，修改和删除要怎么设计?

我会把文档处理做成事件驱动的增量流水线。

新增/修改：文档服务发出变更事件，消费者按 doc_id 解析、切分、生成 embedding，写入新版本；旧版本标记失效。用内容哈希判断是否真的变化，避免无意义重建。

删除：先写 tombstone 并立即从检索过滤中排除，再异步删除向量、缓存和引用；删除要幂等，支持重放。

一致性：维护「文档版本 → 索引版本」的映射；查询时按版本过滤，避免新旧混用。对批量更新做限流和背压，防止把 embedding 和向量库打满。

失败处理：变更事件进队列，失败重试进 DLQ；支持全量重建和断点续传。

可观测：监控积压量、处理延迟、失败率、索引与源数据的一致性。

核心原则是「源数据是真相，索引是可重建的派生数据」，任何索引问题都能通过重建修复。

12. sql query怎么融合到上下文?

把 SQL 当成一路结构化检索，而不是让模型直接拼一段 SQL 文本塞进上下文。

流程是：识别问题需要结构化数据 → 做 Schema Linking（选表、选字段、理解关系）→ 生成 SQL → 校验（语法、只读、权限、行数限制）→ 执行 → 把结果转成表格或结构化摘要 → 和文档检索结果一起组装上下文。

上下文里要保留：SQL、使用的表和字段、查询条件、结果行数、执行时间、数据时间范围。这样模型能解释答案来源，也方便排查。

权限要在 SQL 执行层做，不能让模型看到无权访问的数据；默认只读，禁止 DDL/DML；大结果集要聚合或分页。

如果问题同时需要文档和数据库，我会把两路结果都标注来源，让模型综合，而不是让 SQL 结果覆盖文档结论。

13. rag故障：rag零召回怎么办？reranker效果不好怎么办？召回正确，单没有到LLM怎么办？召回内容冲突或者重复怎么办?

我会按链路分别处理。

零召回：先确认索引是否有数据、embedding 模型是否一致、query 是否超出领域；再检查过滤条件是否过严、阈值是否过高。处理手段包括 query 改写、混合检索、降低阈值、扩大 Top-K、走兜底检索或让模型明确说不知道。

Reranker 效果不好：看标注数据上的 NDCG，判断是候选质量差还是排序模型不匹配；可以换领域模型、减少候选数、调整阈值，或者暂时回退到融合排序。不要在没有评测的情况下频繁换模型。

召回正确但没到 LLM：检查上下文组装和 Token 预算，常见原因是截断、去重误删、排序后被挤出、模板拼接 bug。要把「召回列表」和「最终送给模型的上下文」都记录下来，逐条对比。

冲突或重复：按文档 ID/内容哈希去重；冲突按来源、版本、时间排序并在上下文里显式标注；用 MMR 控制多样性。高风险冲突转人工，不要静默选一个。

14. BadCase怎么做逆向根因分析?

我会用「回放轨迹 + 分段验证 + 5 Why」来做，而不是只看最终答案。

第一步，完整回放：把 query、改写结果、各路召回、过滤、rerank 分数、最终上下文、模型输出都还原出来。

第二步，定位第一处分叉点：从哪一步开始，结果和预期不一致。常见分段是 query 理解 → 检索 → 过滤 → 排序 → 上下文组装 → 生成 → 引用。

第三步，做对照实验：保留其他环节，只替换一个变量。比如用标准 query 重跑、把正确文档强行放进上下文，看模型能不能答对，从而区分「检索问题」还是「生成问题」。

第四步，问 5 Why：为什么没召回？为什么排序低？为什么没进上下文？为什么模型没用？直到找到可执行的修复点。

第五步，归类并修复：数据、切分、索引、检索、排序、Prompt、模型、上下文预算，不同原因对应不同手段。

最后把 case 和根因标签一起回流评测集，验证修复是否真正生效。

15. 怎么换embedding模型?

不能直接替换，因为向量空间不兼容。我会按「离线评测 → 双写双查 → 灰度切换 → 清理旧索引」来做。

第一，离线评测：用带标注的检索集比较新模型和旧模型的 Recall@K、NDCG、延迟和成本，确认新模型在业务语料上确实更好。

第二，双写：新文档同时用新旧模型生成向量，写入两套索引；查询时按开关决定走哪套。

第三，存量重建：对历史文档批量重新 embedding，记录进度、失败和版本；重建期间保持旧索引可用。

第四，灰度：先小流量走新索引，对比效果和延迟；有问题能切回旧索引。

第五，元数据标记 embedding 模型和版本，避免混用；切换完成后保留旧索引一段时间再清理。

如果成本很高，可以先按业务域或文档类型分批切换，不要一次性全量。

16. rag检索为什么还需要重排?

因为粗召回和精排序的目标不同。

向量检索用的是双塔模型，query 和文档分别编码，效率高，但交互信息少，容易召回「语义相近但不真正回答问题」的内容，所以追求的是不漏。

Rerank 用交叉编码器或专门模型，把 query 和候选文档放在一起做精细匹配，能理解更强的相关性信号，追求的是排得准。

工程上通常是：粗召回几十条 → rerank 到几条 → 送给 LLM。这样兼顾召回率和精确率，代价是额外延迟和成本。

如果候选里根本没有正确文档，rerank 也救不回来，所以召回和重排要分开评估。对延迟敏感的场景，可以减少候选数、用轻量 rerank，或者只对 Top-N 做重排。

17. text2sql怎么实现?

完整链路是：问题理解 → Schema Linking → SQL 生成 → 校验 → 执行 → 结果解释。

第一，Schema Linking：从问题里识别表、字段、指标、过滤条件和时间范围，只把相关的 Schema 片段给模型，避免几十张表全塞进去。

第二，SQL 生成：用结构化 Prompt，给出方言、表关系、字段含义和少量示例；优先用受约束生成或工具调用，而不是自由文本。

第三，校验：语法解析、只读检查、表/字段存在性、权限、行数和超时限制；高风险查询要 explain 或 dry run。

第四，执行：用只读账号，参数化查询，限制返回行数；失败时把数据库错误反馈给模型做有限修复。

第五，结果解释：把结果转成表格/摘要，保留 SQL 和数据时间范围，方便引用。

安全上默认只读、禁止 DDL/DML、按租户过滤，所有查询审计。

18. LLM生成Sql怎么保证正确性?

单靠模型不够，要做多层校验。

第一，Schema grounding：只提供相关表结构、字段注释、枚举值和示例，减少模型猜测。

第二，约束生成：用固定方言、模板或 SQL AST 校验；禁止多语句、DDL、全表扫描和超大结果。

第三，执行前校验：语法解析、表字段存在性、权限、行数/超时限制；必要时先 EXPLAIN。

第四，执行后校验：结果为空、异常值、行数异常时不要直接返回，做二次确认或让模型解释。

第五，失败修复：把数据库错误结构化返回给模型，限制修复次数，仍失败就转人工。

第六，高风险场景加人工审核，尤其是涉及金额、权限和敏感数据。

最后用评测集覆盖常见问题类型和边界条件，统计执行成功率、结果正确率和人工修改率。

19. 多轮rag失效怎么办?

多轮 RAG 的核心问题是「当前 query 脱离了上下文」。

第一，做指代消解和 query 重写：把「它」「这个」补全成完整问题，结合上一轮的实体和约束。

第二，维护结构化会话状态：当前主题、已确认条件、待解决问题，而不是把所有历史消息直接拼进检索。

第三，按轮检索：每轮只检索与当前子问题相关的内容；复杂问题先拆解再检索。

第四，处理话题切换：检测到新主题时清理旧上下文，避免旧文档干扰。

第五，证据不足时澄清，而不是重复召回同一批内容。

第六，专门建多轮评测集，覆盖指代、省略、话题切换和条件追加。单轮指标好不代表多轮好。

20. 大流量RAG系统?

我会从「减少请求、分摊压力、保护下游、可降级」四方面设计。

减少请求：精确缓存、语义缓存、Singleflight、相同 query 合并；静态知识预计算。

分摊压力：索引分片和副本、读写分离、检索服务无状态化、按租户限流；embedding 和 rerank 可以异步或批处理。

保护下游：向量库、rerank、LLM 都有超时、熔断和并发上限；入口做准入控制，按预计完成时间拒绝而不是等队列满。

降级：rerank 不可用时退回融合排序；语义缓存不可用时走精确缓存；大模型限流时切小模型或只返回检索摘要；必要时让用户稍后重试。

可观测：P95/P99、召回延迟、rerank 延迟、缓存命中率、上游 429、降级次数。

容量要按峰值和长尾设计，不能只用平均 QPS。大流量下，稳定性比单次效果的微小提升更重要。

21. rag没有怎么让大模型直接说不知道?

要从「指令、检索、判定、评估」四方面一起做。

指令上：明确只能基于给定资料回答，资料不足时输出固定的「不知道/无法回答」结构，并说明缺少什么。

检索上：设相关性阈值和证据充分性检查；召回分数低、文档互相矛盾、关键字段缺失时，直接走拒答，不要把问题丢给模型自由发挥。

判定上：可以加一个轻量的答案校验步骤，检查结论是否有引用支撑；没有证据的结论不允许输出。

评估上：把「该拒答时拒答」作为指标，构造不可回答问题集；不要只看回答率，否则会鼓励模型硬编。

产品上：拒答时给出下一步建议，比如补充哪个条件、换一种问法，而不是只回一句「不知道」。

22. 为什么topK不是越大越好?

Top-K 增大能提高召回率，但会带来四个问题。

第一，噪声增加：不相关文档会干扰模型，降低答案精确率。

第二，上下文稀释：关键信息被大量无关内容淹没，模型更容易忽略中间部分。

第三，冲突变多：不同文档说法不一致，模型难以判断。

第四，成本和延迟上升：Token 更多，rerank 和生成都更慢。

所以 K 应该由评测决定，而不是拍一个固定值。常见做法是：粗召回大一些，rerank 后只取少量高置信内容；或者按分数分布动态截断，分数断崖之后的内容不要。

对多跳问题，可以分步检索，而不是一次取很大 K。关键是「给模型足够且干净的信息」，不是「给得越多越好」。

23. 怎么做意图识别？意图识别错误怎么办？

做法上分三层：规则、模型、兜底。

规则层处理明确命令和高频固定说法；模型层用分类模型或 LLM 处理自然语言变体，输出意图、槽位和置信度；兜底层在置信度低或多意图时澄清，或者走通用检索。

意图 Schema 要定义清楚：名称、描述、典型说法、必填槽位、反例。多意图要支持拆解，而不是强行归到一个类。

识别错误时，先看错误类型：是分类错、槽位缺失，还是用户表达本身有歧义。分类错就补示例和规则；槽位缺失就追问；歧义就给选项。

线上要记录路由决策和用户纠正，定期回流评测集。高风险意图错误不能靠模型自我修正，要有确定性校验和人工兜底。

最后，意图识别要和路由解耦：识别负责「用户要什么」，路由负责「谁来干」，不要把两者混在一个 Prompt 里。

24. 如何保证意图识别的稳定性？

这道题和前面重复，核心还是「规则优先 + 模型补充 + 置信度门控 + 评测闭环」。补充几个稳定性细节：

第一，固定输入格式和上下文顺序，减少无关信息干扰；温度调到接近 0。

第二，Prompt 和意图 Schema 版本化，每次修改都跑回归，避免改一个意图影响其他意图。

第三，对同义表达、错别字、口语化、否定和多意图做专门测试；线上做灰度，观察路由分布是否异常。

第四，置信度要校准，不能直接用模型自报的概率；可以用标注数据做阈值调优。

第五，监控「澄清率、兜底率、用户纠正率」，这些比单纯准确率更能反映稳定性。

25. query改写方法有哪些？各适用于什么场景？

常见有五类。

第一，规范化：去噪、纠错、统一术语、全半角转换，适合用户输入不规范、业务术语多的场景。

第二，指代消解和上下文补全：把「它」「这个」补成完整实体，适合多轮对话。

第三，同义扩展/多查询生成：生成多个表达变体，多路召回后融合，适合口语化和领域术语差异大的场景。

第四，问题拆解：把多跳问题拆成子问题，适合复杂对比、因果、多条件查询。

> 第五，HyDE：先生成假设答案再检索，适合 query 很短、语义稀疏的场景，但要防止假设偏差。
>
> 选择时看成本和风险：规范化和指代消解最安全；多查询会增加检索成本；拆解适合复杂问题；HyDE 要谨慎使用。无论哪种，都要保留原始 query 做兜底，并用评测验证改写是否真的提升召回。

Multi-Agent

1. 为什么要使用Multi-Agent?

多 Agent 主要解决三个问题：上下文隔离、能力专业化、并行执行。

单个 Agent 处理完整测试流程时，需求、用例、执行、日志分析的上下文会互相污染，Prompt 也会变得又长又杂。拆成多个 Agent 后，每个 Agent 只带自己需要的上下文和工具，边界更清晰。不同阶段还可以用不同的模型、权限和验证策略。

另外，某些子任务可以并行，比如多个模块的日志分析、多个环境的检查；多 Agent 能缩短端到端时间。

但多 Agent 不是默认选择。它带来协调成本、错误传播和调试难度。如果单个 Agent 加工具就能做好，或者任务强顺序，我不会拆。判断标准是：上下文是否真的装不下、任务是否能真正并行、是否需要不同的权限边界。

2. MultiAgent的协作模式有什么?

常见有五种。

第一，Supervisor/Orchestrator：一个主 Agent 负责拆解、分派、汇总，子 Agent 只做专业任务。最容易控制和观测，适合大多数生产场景。

第二，Pipeline：按固定阶段串行，上游输出是下游输入，适合流程明确的场景。

第三，Hierarchical：多层主管，适合大型任务，但层级越深越难调试。

第四，Peer/Handoff：平级 Agent 之间转移控制权，适合专家接力，但需要明确交接内容。

第五，Debate/Blackboard：多个 Agent 共享一块状态或互相质疑，适合需要多视角验证的场景，成本和延迟较高。

我通常优先用 Supervisor + 固定/模板化 Workflow，只有在需要动态判断时才引入更复杂的协作模式。协作模式越复杂，越要有统一的追踪和预算控制。

3. MultiAgent带来了什么代价?

主要有五类代价。

第一，协调成本：任务拆解、分派、等待、汇总都要额外逻辑，容易出现职责不清和重复工作。

第二，错误传播：上游 Agent 的一个错误会被下游放大，而且很难判断是谁的责任。

第三，上下文和状态一致性：共享状态容易互相污染，完全隔离又可能丢失必要信息。

第四，成本和延迟：多次模型调用、多次工具调用，Token 和端到端时间都会上升。

第五，调试和评测困难：一次任务跨多个 Agent，没有统一 trace 很难复现和归因。

所以我会控制 Agent 数量，按能力边界拆而不是按角色数量拆；统一状态和消息协议；给每个子任务设预算和超时；用 trace 串起全链路。多 Agent 的收益必须能覆盖这些成本。

4. 多Agent之间上下文怎么管理?

我会遵循「默认隔离、按需共享、结构化交接」。

默认隔离：每个子 Agent 只拿到完成自己任务所需的最小上下文，避免历史消息和无关工具结果污染。

按需共享：通过编排层维护一份最小共享状态，比如任务目标、已完成步骤、关键约束、待办事项；子 Agent 只读写自己负责的字段。

结构化交接：Handoff 时传目标、输入资产、已完成、待办、约束、证据和验收标准，而不是把整段聊天记录复制过去。

汇总时：子 Agent 输出结构化结果，编排层做去重、冲突检测和摘要，再决定给下一个 Agent 什么。

另外要有权限控制：不同 Agent 能访问的记忆和工具不同，不能因为共享状态就绕过权限。

5. 多Agent意见冲突了怎么办?

先区分「事实冲突」和「判断冲突」。

事实冲突：比如两个 Agent 查到不同的配置值。这时回到证据：来源、时间、版本、权限，按可信度排序；无法裁决就标注冲突并转人工，不要平均。

判断冲突：比如两个 Agent 对根因有不同假设。这时让它们各自给出证据和置信度，用确定性规则或验证动作来区分；可以引入 judge Agent，但 judge 也要看证据，不能只靠投票。

工程上：保留每个 Agent 的原始结论和依据；编排层做冲突检测；高风险场景设人工裁决点。

我的原则是：冲突不是坏事，说明系统发现了不确定性；最坏的是静默选一个，让用户以为没有分歧。

6. 为什么是8个Agent，你们是怎么设计区分的?

数量不是目标，边界才是。以 AI4Test 为例，我们是按测试流程的能力边界拆的，大致包括：需求分析、测试设计、用例生成、环境准备、执行、日志分析、缺陷定位、报告汇总，共 8 个角色。

这样拆的依据是：每个阶段有独立的输入输出、工具集和验收标准；上下文可以隔离；部分阶段可以并行；权限也不同，比如环境准备和执行需要写权限，分析和报告只需要读权限。

但 8 不是固定的。如果某个阶段很薄，我会合并；如果某个阶段内部差异很大，我会再拆。关键是每个 Agent 都有清晰的能力标签、输入输出契约和成功标准，而不是为了凑数量。

上线后我会看每个 Agent 的调用频率、成功率和人工修改率；长期没价值或频繁误判的，就合并或下线。

工具调用

1. 工具重复调用怎么办?

先判断是「模型重复决策」还是「系统重复执行」，两者处理方式不同。

模型重复决策：检测相同的工具 + 参数组合，命中就返回缓存结果或提示「已经调用过」，同时在 Prompt 里强调不要重复调用。还可以设最大调用次数、检测连续无进展的循环。

系统重复执行：用幂等键 + 执行记录，保证同一业务操作只产生一次副作用；恢复时先查执行状态，而不是直接重跑。

工程上我会做三件事：按工具名 + 参数哈希做短期去重；把调用历史写进上下文，让模型看到自己已经做过什么；在运行时加循环检测，重复超过阈值就中断或转人工。

重复调用不一定都是错的，比如轮询状态是合理的；所以要去重的是「相同参数的相同动作」，不是所有重复。

2. 工具调用安全怎么保障?

我会做五层防护。

第一，工具白名单：Agent 只能看到和调用被授权的工具，未授权工具不出现在候选里。

第二，参数校验：类型、长度、枚举、路径、URL、SQL、命令都要校验，防注入、路径穿越和 SSRF。

第三，最小权限：默认只读，写权限按需申请；高风险动作动作级审批，不是权限级审批。

第四，沙箱：限制文件、网络、CPU/内存/磁盘、进程数和执行时间；以非 root 运行。

第五，审计与监控：记录谁在什么上下文下调用了什么、参数、结果；异常调用告警。

另外，工具返回也是外部输入，要防 Prompt Injection；不能让工具结果直接触发高风险动作。

底线是：即使模型被完全攻破，它能造成的破坏也被限制在授权和沙箱范围内。

3. 同步Tool和异步Tool怎么处理?

按耗时和是否需要等待结果来分。

同步 Tool：适合毫秒到几秒、结果必须立即用于下一步的操作，比如查询配置、读文件。调用方阻塞等待，设超时和重试。

异步 Tool：适合分钟级、长耗时或有外部副作用的操作，比如跑测试、部署环境、批量分析。调用后立即返回 Job ID，Agent 拿到的是「已提交」状态，后续通过轮询、回调或事件订阅获取结果。

工程上要统一状态机：PENDING → RUNNING → SUCCEEDED / FAILED / CANCELLED；异步任务要有进度、超时、取

消和结果存储。

Agent 侧不能一直阻塞等待，应该把「等待中」作为状态保存，继续做别的或结束当前轮，等结果回来再恢复。

选择原则：能用同步就不要异步，异步会增加状态管理成本；但只要耗时不可控或可能超过请求超时，就必须异步。

4. 长耗时任务如果不设置最大时间，怎么确定工具挂了吗?

不能靠「一直没返回」来判断，要有主动的健康信号。

第一，心跳/进度：工具定期上报心跳或进度，超过 N 个周期没有更新就判定为可疑。

第二，租约：执行方持有租约并续期，租约过期由 Reaper 回收；这比单纯超时更可靠。

第三，外部状态查询：通过任务 ID 查询下游状态，区分「还在跑」「已失败」「已失联」。

第四，分阶段超时：连接、首字节、无进度、总时长分别设阈值，而不是只设一个总超时。

第五，取消与补偿：判定挂起后先尝试取消，再根据幂等和副作用决定是否重试或转人工。

如果工具完全不支持心跳，只能设置保守的总超时，并把它当成「结果不确定」处理，不能假设失败后重跑是安全的。

5. agent调用工具写同一个文件怎么办?

核心是「同一时刻只有一个写者」，不能靠模型自觉。

第一，按文件或资源加锁/租约，拿到锁才能写；锁要有超时和续期，避免死锁。

第二，每个 Agent 在自己的隔离工作区写，最后由编排层合并；冲突时按版本或人工介入。

第三，用乐观并发：写之前检查版本号，版本变了就拒绝并让 Agent 重新读取。

第四，写操作幂等：带业务键和内容哈希，重复写不产生重复内容；关键文件保留版本历史。

第五，明确所有权：编排层指定哪个 Agent 负责哪个文件，其他 Agent 只读。

如果多个 Agent 必须协作写同一文件，最好改成「各自产出片段，由一个合并节点统一写入」，而不是并发直接改。

6. 长时间执行Tool怎么处理?

按异步任务处理，核心是「不占用 Agent 的同步等待」。

第一，工具调用立即返回 Job ID，Agent 记录状态后可以结束当前轮或去做别的事。

第二，任务侧有状态机、进度、心跳、超时和取消；结果落库，支持查询和订阅。

第三，Agent 恢复时先查 Job 状态，完成就回填结果，失败就按错误类型重试或转人工，运行中就继续等待。

第四，设置分阶段超时和无进度检测，避免无限挂起。

第五，结果可能很大，要先摘要或存对象存储，只把关键结果回填上下文。

第六，用户体验上要给进度反馈，长任务不能看起来像卡死。

如果工具支持回调，优先用回调/事件，而不是高频轮询；轮询要有退避，避免打爆下游。

7. 工具Timeout怎么设计?

我会分四层设置：连接超时、首字节/首包超时、无进度超时、总超时。

连接超时防止网络不可达；首包超时防止下游接受连接但不处理；无进度超时用于长任务，超过一定时间没有心跳就判定异常；总超时是最后兜底。

超时值按工具类型配置，不是全局一个数：查询类可以短，执行类要长；高风险或不可逆操作要留足时间，避免超时后状态不确定。

超时要能向上游传播（deadline propagation），不要每一层都重新计时，否则总时间会被放大。

超时后要取消上游请求，避免继续消耗资源；返回结构化错误，让 Agent 知道是超时而不是业务失败。

最后，超时和重试要配合：只对幂等、可重试的操作重试，并且总预算不能超过用户可接受时间。

8. 不同工具retry策略怎么设计?

先给工具分类，再定策略。

只读/幂等工具：网络错误、超时、429、5xx 可以重试，指数退避加抖动，设最大次数和总预算。

有副作用但支持幂等的工具：可以重试，但必须用同一个幂等键，并先查执行状态，避免重复副作用。

有副作用且不支持幂等的工具：默认不自动重试；失败后查状态、补偿或转人工。

参数/业务错误：不重试，直接把明确原因返回给模型或用户。

工具不可用：熔断 + 切备用工具/降级，而不是原地重试。

所有重试要收敛到一层，避免 SDK、网关、业务层同时重试导致风暴；还要有全局重试预算和告警。

9. tool结果太大怎么办?

不能把原始大结果直接塞回上下文。

第一，结构化抽取：只保留当前任务需要的字段，比如状态、错误码、关键数值、前 N 条记录。

第二，摘要：让工具或轻量模型生成摘要，保留结论和证据位置；原始结果存对象存储，上下文里放引用。

第三，分页/分段：需要时再取下一段，而不是一次全给。

第四，截断并标注：设硬上限，超出部分明确标记「已截断」，避免模型误以为看到了全部。

第五，去重和过滤：去掉重复行、无关字段和噪声。

第六，按任务阶段取数：不同阶段需要的信息不同，不要一次性把所有结果都注入。

关键是让模型知道「这是摘要/部分结果」，并把原始数据保留下来供追溯。

10. tool执行过程中Agent挂了怎么办?

靠检查点 + 执行记录 + 幂等恢复。

第一，调用工具前写检查点：记录工具名、参数、幂等键、步骤状态。

第二，调用发出后标记为 `IN_FLIGHT`；拿到结果后写 `SUCCEEDED` 和结果；失败写 `FAILED`；超时无回执标记 `UNCERTAIN`。

第三，`Agent` 恢复时先读检查点，再核对工具执行记录：成功就回填结果，失败且可重试就重试，不确定就查询下游或转人工。

第四，工具本身尽量幂等，保证重放安全。

第五，租约机制保证同一任务不会被多个 `Worker` 同时执行。

核心是不能假设「进程挂了 = 工具没执行」。有副作用的调用必须显式处理「结果不确定」这个状态。

11. tool执行成功后agent如何继续?

工具成功后，系统要做三件事：持久化结果、回填上下文、推进状态。

第一，把结果写入执行记录和检查点，标记工具调用成功；这样即使 `Agent` 马上崩溃，恢复时也不会重复执行。

第二，把结果转成结构化的工具消息回填给模型，包含状态、数据、错误码（如果有）和必要摘要；超大结果只回填摘要和引用。

第三，更新任务状态和计划：标记当前步骤完成，检查依赖是否满足，决定下一步是继续调用工具、生成答案还是等待人工。

如果工具成功但结果语义不完整，不能直接让模型猜，要先做校验/补全，再回填。

恢复场景下，如果发现工具已成功但结果没回填，直接从执行记录里取结果继续，而不是重新调用。

12. 工具选择错误怎么办?

先分析错误类型：是工具描述不清、候选太多、模型理解错，还是路由/权限问题。

第一，优化工具描述：写清楚「做什么、什么时候用、什么时候不要用」，名字用动词开头，减少相似工具的歧义。

第二，缩小候选集：按意图、权限、场景先过滤，不要一次给模型几十个工具；相似能力可以合并或加命名空间。

第三，加示例和反例：在 Prompt 或工具描述里给出典型调用场景。

第四，加校验：选错工具往往伴随参数不匹配，用 Schema 和业务规则拦截。

第五，兜底：置信度低时让模型澄清，或者先走通用检索/默认工具；高风险操作必须人工确认。

第六，用评测集统计工具选择准确率，按工具和场景分析，持续优化。不要只靠换更大的模型。

13. 工具参数过多怎么办?

参数过多会增加模型填错、漏填和幻觉的概率。

第一，拆工具：把「大而全」的工具按动作拆成多个小工具，每个只保留必要参数。

第二，用嵌套对象分组：把相关参数收进一个 config 对象，减少顶层参数数量；但嵌套层次不要太深。

第三，设默认值和枚举：能默认的默认，能枚举的不要自由文本；必填参数尽量少。

第四，渐进式披露：先让模型选工具和关键参数，缺失的再追问或由系统补全。

第五，参数校验和修复：Schema 校验、类型转换、有限自愈，失败时返回明确错误。

第六，如果参数确实很多，考虑用配置文件/模板，让模型只填少量变量。

原则是：工具接口按「模型容易用对」设计，而不是按「后端函数签名」直接暴露。

14. tool schema怎么版本化?

把 Tool Schema 当成对外 API 契约来管理。

第一，语义化版本：主版本不兼容、次版本新增可选字段、补丁修描述；Agent 配置要固定使用的版本。

第二，注册表：集中管理工具名、版本、输入输出 Schema、权限、风险等级和负责人。

第三，向后兼容：新增字段尽量可选，不要随意改字段名或类型；废弃字段要保留过渡期并标注。

第四，契约测试：每次变更跑 Schema 校验和调用测试，确保旧 Agent 还能用。

第五，灰度与回滚：新版本先给小流量或新 Agent 用，观察调用成功率；出问题能切回旧版本。

第六，审计：记录哪个 Agent、哪个任务用了哪个版本，方便归因。

工具升级不能悄悄改变语义，否则模型行为会不可预测。

15. 为什么agent tool必须考虑幂等?

因为 Agent 的执行环境天然会出现重复：网络超时后重试、进程崩溃后恢复、消息重复投递、租约过期后任务被重新分配。

如果工具没有副作用，重复执行最多浪费资源；但只要有写操作，比如创建订单、改配置、发消息、写文件，重复执行就会产生真实损失。

更麻烦的是「结果不确定」：请求发出去了，但没收到回执。系统不知道到底执行没有，如果不幂等，重试和不重试都可能错。

所以幂等不是可选项，而是有副作用工具的准入条件。它让重试变安全，让恢复有依据，也让「至少一次」的投递模型可以落地。

对无法幂等的工具，就必须用审批、锁、人工确认或补偿来兜底，不能假设不会重复。

16. 怎么实现tool幂等?

核心是「幂等键 + 状态机 + 原子占位 + 结果缓存」。

第一，调用前用业务唯一键生成幂等键，比如租户 + 任务 + 步骤 + 工具名，保证同一逻辑操作重试用同一个键。

第二，原子地检查并创建记录：不存在就置为 PROCESSING；已存在就看状态。用 Redis Lua 或数据库唯一约束保证原子性。

第三，状态为 SUCCESS 时直接返回缓存结果；PROCESSING 时返回处理中或等待；FAILED 时根据错误类型决定是否允许重试。

第四，执行完成后写 SUCCESS/FAILED 和结果，设置合理 TTL，覆盖业务重试窗口。

第五，下游如果支持幂等接口，把幂等键透传过去；不支持就自己做查询/补偿。

第六，记录调用轨迹，方便排查「为什么没有重复执行」或「为什么被判定为重复」。幂等只保证重复安全，不保证第一次一定成功。

17. 幂等建怎么设计?

幂等键要满足「同一业务操作稳定、不同操作可区分」。

第一，用业务语义生成，而不是随机 UUID。常见组合是：租户 ID + 任务 ID + 步骤 ID + 工具名，必要时再加业务实体 ID。

第二，同一个逻辑操作在重试、恢复、重复投递时必须生成同一个键，所以键不能包含时间戳、随机数或每次变化的参数。

第三，作用域要明确：全局唯一还是租户内唯一；太宽会冲突，太窄会重复。

第四，键要可读、可排查，必要时加哈希；长度受存储限制，可以对长参数做稳定哈希。

第五，设置合理 TTL，覆盖业务重试和人工处理窗口；太短会失去幂等，太长会占用存储。

第六，记录键与结果的映射，支持查询和审计。

最后，幂等键只是入口，真正的安全还依赖状态机和下游的幂等能力。

18. agent崩溃后如何避免重复执行?

靠「检查点 + 工具执行记录 + 租约 + 幂等」四件套。

第一，每一步和每次工具调用前写检查点，记录当前步骤、参数和状态。

第二，工具调用有独立执行记录：IN_FLIGHT、SUCCEEDED、FAILED、UNCERTAIN；结果也要落库。

第三，租约保证同一时刻只有一个 Worker 处理该任务；Worker 崩溃后租约过期，任务被回收。

第四，恢复时先读检查点，再核对工具记录：成功就回填结果并继续；失败且可重试才重试；UNCERTAIN 先查询下游或转人工。

第五，所有有副作用的工具尽量幂等，保证重放安全。

核心是：不要假设「崩溃 = 没执行」。恢复逻辑必须能处理「已执行但结果未回填」的中间状态。

19. tool本身无法执行幂等怎么办?

不能自动重试，要换一套策略。

第一，调用前先查询：如果下游提供状态查询接口，先查这个业务操作是否已经完成。

第二，加锁/租约：保证同一资源同一时刻只有一个执行者，减少并发重复。

第三，用「至多一次」语义：失败后不自动重试，标记为不确定，转人工确认或补偿。

第四，补偿事务：如果重复执行造成了副作用，提供反向操作，比如取消、删除、退款。

第五，Outbox/事件：把「要执行」和「已执行」记录下来，通过可靠事件和消费端去重降低重复概率。

第六，审批：高风险且不可幂等的操作必须人工确认。

如果既不能幂等、又不能查询、还不能补偿，那这个工具就不应该交给 Agent 自动执行。

20. 工具调用失败怎么办?

先分类，再决定处理方式。

参数错误：把校验错误返回给模型修复一次，仍失败就转人工。

瞬时故障：连接失败、超时、429、5xx，可以重试，指数退避加抖动，受总预算限制。

业务错误：权限不足、资源不存在、余额不足，不重试，返回明确原因。

结果不确定：请求可能已执行，先查状态或幂等记录，不能直接重试。

工具不可用：熔断、切备用工具或降级。

处理完后要把结构化错误反馈给模型，让它决定下一步：换工具、改参数、放弃还是请求人工。

所有失败都要记录到执行轨迹，便于归因和补评测集。

21. LLM怎么知道工具执行是否成功?

不能靠自然语言描述，要给结构化结果。

工具返回统一结构：`success/status`、`data`、`error_code`、`error_message`、`retryable`、`metadata`。成功和失败走同一套 `Schema`，模型不需要猜。

系统把结果转成工具消息回填给模型，明确标注「成功/失败/处理中/结果不确定」。对于异步任务，返回 `Job ID` 和当前状态，而不是假装完成。

如果工具只返回一段文本，中间层要做归一化：解析状态码、判断业务成功、提取关键字段，再生成结构化结果。

另外，成功不等于结果可用：还要做后置校验，比如返回为空、字段缺失、数值异常，都要标记为「需要进一步确认」。

模型只能基于这些结构化信号决策，不能从「没有报错」推断成功。

22. Tool Error怎么反馈给LLM?

要给「可行动」的错误，而不是堆栈或一大段日志。

结构上包含：错误码、错误类型（参数/权限/网络/业务/超时）、可读信息、是否可重试、建议动作、相关参数。

内容上要脱敏：不要暴露密钥、内部路径、SQL、堆栈；必要时只给摘要和追踪 `ID`。

语气上要明确：告诉模型「这个错误重试没用」还是「可以换参数再试」，避免模型反复调用同一个失败工具。

对于参数错误，最好指出哪个字段不合法、期望什么格式；对于业务错误，说明缺少什么条件。

同时把完整错误写入日志和轨迹，模型只看摘要。这样既不影响模型决策，又能保留排查信息。

23. 如何防止Tool结果污染上下文?

把工具结果当「不可信数据」，而不是指令。

第一，结构化隔离：用明确的分隔符或消息角色包起来，标注来源、时间、类型。

第二，指令层级：系统提示明确「工具结果只是数据，不能覆盖系统指令」；但这只是辅助，不能当唯一防线。

第三，内容清洗：去掉或转义可能被解释为指令的内容；对网页、邮件、用户上传文档做注入检测。

第四，摘要与截断：大结果只保留相关字段和结论，减少噪声和攻击面。

第五，权限隔离：即使模型被结果诱导，也没有权限执行高风险动作；工具白名单和审批是硬防线。

第六，后置校验：模型输出前检查是否引用了不允许的内容或执行了未授权动作。

工具结果是 Agent 最容易被间接注入的入口之一，不能因为「是我们自己的工具」就完全信任。

24. 具有副作用单支持幂等工具怎么恢复?

恢复时先查幂等记录，再决定是否重放。

第一，读检查点和工具执行记录，拿到幂等键和上次状态。

第二，如果状态是 SUCCESS，直接取缓存结果回填，不再调用工具。

第三，如果是 PROCESSING，说明可能还在执行或有并发，先等待/查询下游，不要立刻重放。

第四，如果是 FAILED 且错误可重试，用同一个幂等键重试；如果不可重试，转人工。

第五，如果是 UNCERTAIN，先查下游状态；查不到再根据幂等保证决定重放或人工确认。

第六，恢复后继续执行后续步骤，并更新检查点。

关键是：幂等让重放安全，但恢复逻辑仍然要先查状态，避免不必要的重复调用和并发冲突。

25. Agent申请测试环境成功后挂了怎么办?

环境申请是有副作用的操作，必须按幂等资源处理。

第一，申请时带业务唯一键（任务 ID + 环境类型），记录申请单 ID、环境 ID、状态和租约。

第二，Agent 挂了之后，恢复逻辑先查申请单/环境状态：如果已成功，直接复用，不重复申请；如果处理中，等待或查询；如果失败，按错误类型重试。

第三，环境要有 TTL 和归属信息，避免孤儿环境长期占用资源；任务取消或完成后触发回收。

第四，如果无法查询状态，标记为不确定，转人工确认，不能盲目再申请一个。

第五，所有操作审计，记录谁申请、什么时候、用途、释放时间。

这类问题的本质是「资源生命周期管理」，不能只当成一次工具调用。

26. 工具重试除了设置最大调用次数，还有什么机制?

还有六类机制。

第一，错误分类：只重试瞬时、可重试的错误，参数/业务错误不重试。

第二，退避与抖动：指数退避加随机抖动，避免重试风暴和同时重试。

第三，全局预算：按请求、租户、模型设 Token/时间/成本上限，超过就快速失败。

第四，熔断：错误率或延迟超阈值就暂时停止调用，给下游恢复时间。

第五，幂等与状态查询：重试前先确认操作是否已经成功，避免重复副作用。

第六，DLQ 与告警：多次失败进入死信队列，人工处理并触发告警。

另外还有超时传播、并发限制、优先级调度和降级备用工具。重试不是一个计数器，而是整套失败处理策略的一部分。

27. 如果工具调用是成功的，但结果反馈语义不完整，模型容易误判，怎么设计中间层?

中间层要做「归一化 + 校验 + 补全 + 置信度」。

第一，归一化：把不同工具的返回统一成结构化结果，明确 status、data、error、metadata，不要让模型解析自由文本。

第二，后置校验：检查必填字段、业务约束、数值范围、时间有效性；发现缺失就标记为「不完整」，而不是当成成功。

第三，补全：能通过二次查询、关联数据或默认规则补全的就补全；不能补全的明确告诉模型缺什么。

第四，摘要与证据：给模型结论性摘要，同时保留原始结果引用；关键结论要求引用具体字段。

第五，置信度：对不完整或来源不可靠的结果给低置信度，触发追问或人工确认。

第六，中间层最好是确定性的，不要再用一个 LLM 去猜另一个 LLM 的结果。

目标是让模型看到「可判断的结构化事实」，而不是一段需要猜的文本。

28. 同一个能力，是做成”大而全“，还是”小而多“工具，怎么权衡?

我倾向于「小而多、可组合」，但有条件。

小而多的优点：职责单一、描述清晰、参数少，模型更容易选对、填对；权限和风险也更好控制；单个工具可以独立测试和

复用。

缺点是工具数量多，模型选择成本上升，容易出现相似工具混淆。

大而全的优点是入口少、调用简单；缺点是参数多、语义模糊、权限难细分，模型容易误用。

我的权衡标准是：
1. 动作差异大、风险等级不同 → 拆开；
2. 参数高度重叠、总是成组使用 → 可以合并；
3. 工具数量超过模型稳定选择的范围 → 加命名空间、路由或分组；
4. 有副作用的高风险操作 → 尽量独立，便于审批和审计。

最终用「工具选择准确率、参数正确率、调用次数」来评估，而不是凭感觉。

29. 如何设工具schema，降低模型幻觉?

用 Schema 把「自由发挥」变成「受约束选择」。

第一，参数尽量用枚举、布尔、数字和固定格式，少用自由文本；必填参数最小化。

第二，字段名和描述要具体，说明单位、格式、取值范围和示例；不要用模糊的「data」「config」。

第三，工具描述写清楚适用和不适用场景，减少选错工具。

第四，用嵌套对象组织相关参数，但层次不要太深；避免几十个顶层参数。

第五，提供少量高质量示例，覆盖典型和边界调用。

第六，服务端做严格校验和有限修复；校验失败返回具体字段错误，而不是让模型猜。

第七，能用结构化输出/约束解码的就用，不要只在 Prompt 里写「请输出 JSON」。

Schema 越清晰，模型需要「编」的空间越小，幻觉自然下降。

30. Agent做多轮工具调用和单轮相比有哪些挑战?

主要有六个挑战。

第一，状态管理：多轮要记住已经调过什么、结果是什么、当前计划到哪一步；状态丢了就会重复调用或漏步骤。

第二，上下文膨胀：工具结果一轮轮累积，容易超限或稀释关键信息，需要摘要、裁剪和外部存储。

第三，错误恢复：中间某一步失败，要判断是重试、换路径还是回滚，不能整条链重来。

第四，依赖和顺序：后面的调用可能依赖前面的结果，不能并行或乱序。

第五，循环和预算：模型可能反复调用同一个工具，需要最大步数、重复检测和成本预算。

第六，可观测性：链路长，必须用 `trace` 串起来，否则出错无法定位。

单轮更像一次函数调用，多轮更像一个分布式流程，可靠性和状态管理是主要难点。

31. 多工具场景下怎么定义工具调用的优先级?

优先级不能只让模型自由决定，要有确定性策略。

第一，按风险和副作用分级：只读查询优先，写操作其次，高风险操作最后且需要审批。

第二，按依赖关系：有数据依赖的工具必须串行，上游没完成不能调下游。

第三，按成本和时延：便宜、快的工具先试，比如先查缓存/本地索引，再调外部服务。

第四，按业务规则：某些场景有明确的工具顺序，比如先校验权限再执行。

第五，按置信度：模型对工具选择不确定时，先走通用/只读工具，或澄清。

工程上可以给工具加元数据：风险等级、成本、依赖、优先级；编排层据此排序，模型只负责提议。

最终用执行成功率和成本评估优先级策略，不要把所有判断都压给模型。

记忆机制

1. agent记忆分类?

我会分四层：工作记忆、会话记忆、长期记忆、经验记忆。

工作记忆是当前这一步需要的信息：目标、当前计划、最近的工具结果，放在上下文里。

会话记忆是这一轮对话的历史和状态：用户说过什么、确认了什么、待办是什么，可以落库。

长期记忆是跨会话的信息：用户偏好、项目事实、业务规则、身份信息，需要检索。

经验记忆是成功/失败案例和复盘结论：什么场景用什么方法有效，用于以后复用。

还可以按归属分：用户记忆、`Agent` 私有记忆、团队/共享记忆。不同层的生命周期、存储和权限都不一样，不能混在一起。

2. 记忆的生命周期?

一般分六个阶段：写入、 `consolidation`（整理/合并）、检索、使用、衰减、删除。

写入：从对话、任务结果或用户显式指令中提取候选记忆。

整理：去重、合并、补元数据、判断置信度；低置信度的先放候选区。

检索：按相关性、时效、重要性、归属召回，注入上下文。

使用：模型基于记忆做决策，记录哪些记忆被使用。

衰减：按时间、重要性、使用频率降权；过期记忆标记失效。

删除：用户要求删除、隐私合规、错误记忆纠正时，级联删除原文、向量、缓存和派生记忆。

生命周期要有明确策略，否则记忆会越存越脏。

3. 记忆应该存哪里?

按类型和访问模式选，不是所有记忆都进向量库。

工作记忆：上下文/内存，生命周期最短。

会话记忆：`Redis` 或数据库，按 `session ID` 查询，支持 `TTL`。

结构化长期记忆：关系型数据库，比如用户偏好、配置、事实字段，需要精确查询和更新。

非结构化长期记忆：向量库，存对话摘要、文档片段，按语义召回。

原始数据和大对象：对象存储，向量或数据库只存引用。

经验记忆：可以结构化 + 向量混合，既能按标签过滤，也能按语义检索。

实际系统通常是组合，并且要有统一的记忆 ID、来源、版本和权限。不要把所有东西都塞进一个向量库。

4. 记忆应该怎么存?

每条记忆至少要有：内容、类型、来源、时间、置信度、归属、有效期、版本。

内容：结构化字段 + 自然语言描述；能结构化的不要只存一段文本。

来源：来自哪次对话、哪个任务、哪个文档，支持追溯。

时间：创建时间、最后更新时间、过期时间。

置信度：用户明确确认的高，模型推断的低。

归属：租户、用户、Agent、项目、会话，决定检索过滤和权限。

版本：更新不覆盖，保留历史，支持冲突处理和回滚。

存储时还要做去重（内容哈希/语义相似）和索引（关键词 + 向量 + 结构化字段）。记忆不是日志，不能只追加不治理。

5. 记忆什么时候存？谁来存?

分三种来源。

用户显式：用户说「记住这个」，直接写入，置信度高；写入前确认内容，避免误解。

Agent 主动：任务完成后，把确认过的偏好、事实、决策写入；只写对以后有用的，不写临时状态。

系统触发：从任务结果、人工纠正、复盘记录中提取；这类先作为候选，经过验证再转正。

谁来存：不要让模型完全自由决定。我会用「提取器 + 规则 + 人工确认」：模型负责提议，系统负责去重、权限和置信度，关键记忆由用户或业务确认。

写入时机也很重要：过程里少写，关键节点写；临时信息带 TTL，长期信息才持久化。

原则是「宁可少写、写准」，记忆污染比没有记忆更难修。

6. 8种记忆原子操作?

我理解是记忆系统的基础操作，可以归纳为八种：

1. Create：新增一条记忆，带来源、置信度、归属。
2. Read/Retrieve：按语义、关键词、元数据召回。
3. Update：修正内容或元数据，保留版本。
4. Merge：把重复或相关的多条记忆合并成一条。
5. Summarize：把长历史压缩成摘要，保留关键决策和约束。
6. Link：建立记忆之间、记忆与实体/任务之间的关系。
7. Decay/Forget：按时间、重要性、使用频率降权或标记失效。
8. Delete：物理或逻辑删除，级联清理向量、缓存和派生数据。

不同系统可能叫法不同，但核心都是「增删改查 + 合并 + 总结 + 关联 + 遗忘」。把这八种操作定义清楚，记忆才不会变成只写不读的黑洞。

7. 记忆如何遗忘?

遗忘不是简单删除，要分策略。

第一，TTL：临时记忆到期自动失效，比如会话状态、短期偏好。

第二，重要性衰减：长期不用、低置信度、低价值的记忆逐步降权，检索时排后或过滤。

第三，冲突失效：被新版本或用户纠正覆盖的旧记忆标记为失效，保留历史但不参与检索。

第四，LRU/容量淘汰：存储有上限时，优先淘汰最久未使用、价值最低的。

第五，用户删除：用户要求删除时，级联删除原文、向量、缓存、备份中的相关数据，并记录审计。

第六，合规删除：按数据保留策略定期清理。

遗忘要和检索过滤配合：标记失效的记忆不能继续被召回。否则「遗忘了」只是表面状态。

8. 记忆如何总结?

总结要保留「决策、约束、待办、事实」，而不是只压缩语气。

第一，增量总结：每 N 轮或上下文到阈值时，把新增内容合并进已有摘要，避免每次全量重写。

第二，分层总结：会话级摘要 → 任务级摘要 → 长期记忆；不同层保留不同粒度。

第三，结构化输出：目标、已确认事实、约束、决策、未完成事项、关键证据，分字段保存。

第四，保留来源：摘要里的关键结论要能链接回原始消息，防止信息丢失后无法追溯。

第五，评测：用「总结后能否回答关键问题」来评估，而不是看摘要像不像。

第六，关键任务少压缩或不压缩，原文单独存储。

总结是损失信息的过程，不能把摘要当成唯一事实来源。

9. 业务相关记忆字段?

除了通用字段，我会加业务维度的信息。

身份与范围：租户、用户、角色、部门、项目、任务 ID。

业务实体：客户、订单、设备、环境、测试用例、版本号等，方便按实体检索和关联。

业务属性：偏好、配置、阈值、约束、历史决策、常用参数。

权限与密级：哪些角色可读、是否敏感、是否可跨租户共享。

时效：业务有效期、版本、最后确认时间。

来源：哪次对话、哪个任务、哪个文档、谁确认的。

状态：候选/已确认/已失效/已删除。

这些字段决定了记忆能不能被正确过滤、追溯和治理。只存一段自然语言，后面很难用。

10. 多条记忆合并?

先判断是「重复」还是「相关但不同」。

重复记忆：内容哈希或语义相似度超过阈值，保留最新、置信度最高的一条，合并来源和使用记录。

相关记忆：比如同一用户的多条偏好，可以合并成一条结构化记忆，字段取并集；冲突字段保留版本和来源，不要直接覆盖。

合并策略：
1. 按实体/主题聚合；
2. 按时间和来源排序；
3. 冲突时以用户确认、最新、权威来源优先；
4. 合并后保留原始记忆的引用，支持回溯；
5. 合并操作本身要可审计、可撤销。

不要为了「简洁」把不同场景的记忆硬合并，否则会丢上下文。合并的目标是减少重复和冲突，不是让存储看起来干净。

11. 记忆和多智能体怎么结合?

我会分三种作用域：Agent 私有、任务共享、团队/全局。

Agent 私有记忆：只存这个 Agent 的专业经验，比如排障 Agent 的诊断模式；其他 Agent 不直接访问。

任务共享记忆：当前任务的目标、状态、已确认事实、证据，通过编排层或黑板共享；所有子 Agent 只读写自己负责的字段。

团队/全局记忆：跨任务的业务规则、用户偏好、组织知识，需要权限控制和冲突治理。

结合方式：子 Agent 开始时按需检索共享记忆；结束时输出结构化结果，由编排层决定写回哪一层。

关键是权限和污染控制：不能让一个 Agent 的错误结论直接写进全局记忆；共享记忆要有来源、置信度和审核。

12. 共享记忆是什么，有什么作用?

共享记忆是多个 Agent 或用户共同可见、可复用的一份记忆，通常按团队、项目或任务范围组织。

作用有三个：
1. 减少重复：同一个事实不用每个 Agent 都重新查一遍；
2. 保持一致性：大家基于同一份目标、约束和已确认事实工作；
3. 支持协作：Handoff 时不用把全部上下文复制过去，接手的 Agent 能直接读共享状态。

但共享记忆也有风险：错误会传播、权限容易越界、并发写入会冲突。

所以我会给它加：作用域（团队/项目/任务）、权限、版本、来源、置信度和写入审批。共享记忆不是「所有人都能写的一块黑板」，而是有治理的共享状态。

13. memory scope怎么设计?

scope 决定「这条记忆谁能看、用多久、和谁共享」。

常见层级：全局 → 租户 → 用户 → 项目/任务 → 会话 → Agent。

设计时要回答三个问题：
1. 归属：这条记忆属于谁，检索时必须带哪些过滤条件；
2. 生命周期：跟会话走、跟任务走，还是长期保留；
3. 权限：哪些角色/Agent 可读、可写、可删。

工程上把 scope 作为必填元数据，检索和写入都强制校验；缓存 key 也要带 scope，避免跨范围命中。

默认最小 scope，需要共享时显式提升；跨 scope 访问必须授权并审计。

scope 设计不清，后面一定会出现越权、污染和删除不干净的问题。

14. 记忆存储怎么选，怎么设计?

按访问模式和数据类型选，通常是组合方案。

工作/会话记忆：Redis，低延迟、TTL 方便，适合临时状态。

结构化长期记忆：PostgreSQL/MySQL，字段明确、支持事务、精确查询和权限过滤。

语义记忆：向量库，支持相似度检索；元数据过滤要包含租户、用户、scope。

原始内容/大对象：对象存储，数据库只存引用。

经验/案例：结构化表 + 向量索引，既能按标签过滤，也能按语义召回。

设计上统一记忆 ID、来源、版本、置信度、TTL 和权限；写入走统一服务，避免各模块直连存储；删除要级联。

不要用一个向量库存所有记忆，结构化查询、事务和权限都会很难做。

15. 记忆与agent进化?

记忆是 Agent 进化的原材料，但不能让模型自由改自己。

流程是：积累经验 → 复盘归因 → 抽取可复用模式 → 形成技能/规则 → 验证 → 上线 → 继续收集反馈。

经验记忆保存「什么场景、做了什么、结果如何、为什么」；复盘时区分偶然成功和可复用规律；抽取成结构化技能、Prompt 片段、工具使用策略或检查清单。

验证很重要：新技能先离线评测，再小流量灰度，不能因为一次成功就写进全局。

还要防止过拟合和污染：保留反例，设置置信度和适用范围，允许回滚。

我理解的 Agent 进化是「受控的经验沉淀」，不是模型在线自我修改权重。

16. chatgpt记忆机制?

我理解 ChatGPT 的记忆大致分两类：用户显式保存的记忆，以及从历史对话中提取的长期偏好/事实。

用户可以在设置里查看、修改、删除记忆；模型在后续对话中会把相关记忆作为上下文注入。它不是把全部聊天记录永久塞进模型，而是保存摘要或结构化要点，按相关性使用。

工程上值得借鉴的是：用户可见可控、记忆与对话分离、支持删除、按相关性检索。

但具体实现细节官方没有完全公开，所以回答时我会区分「已知的产品行为」和「推测的实现方式」，不把推测说成事实。

如果面试官追问，我会讲自己会怎么设计：显式记忆 + 候选记忆 + 用户确认 + 作用域 + 删除级联。

17. 用户突然删除一条对话记录，相关记忆上库了怎么办?

要能级联删除派生记忆，而不是只删原始对话。

第一，建立来源链路：每条记忆记录来自哪条对话/消息，支持反查。

第二，删除对话时触发级联：删除由该对话提取的记忆、摘要、向量、缓存和关联索引；不能只删原文。

第三，异步一致性：删除是异步的，但要有 tombstone 立即阻止检索；完成后清理物理数据。

第四，处理共享/派生：如果记忆被合并或共享，要判断是否还有其他合法来源；没有就删除，有就保留并更新来源。

第五，审计：记录删除请求、范围、执行结果，满足合规要求。

第六，备份/日志：按保留策略处理，不能嘴里说删了、备份里还有。

这类问题本质是数据治理和隐私合规，设计记忆时就要把「可追溯、可删除」作为一等需求。

上下文管理

1. 上下文压缩方法有哪些?

常见有七类。

第一，滑动窗口：只保留最近 N 轮，简单但会丢早期信息。

第二，摘要：把历史对话压缩成摘要，保留目标、决策、约束和待办。

第三，分层摘要：会话级 → 任务级 → 长期记忆，不同粒度。

第四，任务状态压缩：用结构化槽位保存目标、已完成、待办、约束，替代冗长历史。

第五，检索式上下文：历史放外部存储，按当前任务检索注入。

第六，工具结果裁剪：只保留结论和关键字段，原始结果放外部。

第七，去重和过滤：去掉重复、寒暄、无关内容。

选择时看任务：短对话用窗口，长任务用状态 + 摘要，多轮问答用检索。压缩一定会丢信息，所以关键事实要独立存储。

2. 压缩如何保证真实不丢失?

压缩不可能完全无损，但可以做到「关键事实不丢、可追溯」。

第一，原文不删：压缩结果进上下文，原始消息和工具结果落库，需要时能回查。

第二，结构化保存关键信息：目标、约束、决策、待办、已确认事实单独存字段，不依赖摘要。

第三，优先抽取式压缩：先保留原文关键句，再做改写；比纯生成式摘要更可控。

第四，保留来源：摘要中的结论链接到原始消息或文档，支持核对。

第五，压缩后校验：用一组关键问题测试摘要能否回答，检查是否丢了否定、条件、数字。

第六，高风险任务少压缩或不压缩，或者压缩后要求人工确认。

最后，让模型知道「这是摘要，不是原文」，避免它把不确定的概括当成事实。

3. 压缩后怎么不忘用户重要需求?

把重要需求从「历史消息」提升为「结构化状态」。

第一，首次出现时就提取：目标、硬约束、偏好、截止时间、禁止事项，写入任务状态。

第二，每轮都重新注入：在系统提示或任务状态里固定带上这些字段，而不是指望模型从摘要里回忆。

第三，优先级标记：硬约束 > 用户偏好 > 一般背景；冲突时以硬约束为准。

第四，用检查清单：任务结束前逐条核对是否满足。

第五，压缩时保护这些字段：不允许摘要把它们合并掉或改写成模糊表述。

第六，评测：构造「中途插入重要约束，再经过多轮压缩」的测试，看模型是否还记得。

关键需求不能只存在于对话历史里，必须有独立、稳定的存储和注入机制。

4. 长上下文里，怎么让agent不忘记关键信息?

不能只靠「上下文长」，要靠结构和机制。

第一，关键信息前置或重复：系统指令、目标、硬约束放在开头，必要时在中间和结尾重复提醒。

第二，结构化状态：目标、待办、已确认事实单独维护，每步注入。

第三，分段处理：长文档先摘要/索引，按需检索，而不是整篇塞入。

第四，外部记忆：把重要信息写入记忆或文件，需要时读取，减少对上下文记忆的依赖。

第五，检查点：每个阶段结束时保存状态，恢复时从检查点继续。

第六，验证：让模型在关键步骤复述目标和约束，或者用规则检查输出是否违反。

长上下文仍有「中间信息被忽略」的问题，所以结构化和外部记忆比单纯堆长度更可靠。

Agent运行时

1. 异常检测怎么实现?

我会分三层：规则、统计、语义。

规则层：最大步数、超时、预算、连续相同工具调用、输出 Schema 校验、错误码阈值。这些确定性高、成本低。

统计层：延迟、Token、工具失败率、任务成功率相对基线的偏移；用滑动窗口和阈值告警。

语义层：用 LLM 或小模型判断「是否在原地打转」「是否答非所问」「计划是否偏离目标」，适合规则覆盖不了的异常。

工程上要先把执行轨迹结构化：每步的输入、输出、工具、耗时、状态。没有轨迹，异常检测只能看最终结果。

检测到异常后的动作也要分级：记录、告警、重试、降级、中断、转人工。不能一发现就杀任务。

最后用历史故障样本评估误报和漏报，持续调阈值。

2. 为什么异常检测需要LLM?

因为很多异常是语义层面的，规则很难枚举。

比如：Agent 连续几步都在「查资料」，但方向和目标越来越偏；工具都成功了，但结论和证据不匹配；用户的需求被逐渐忽略。这些从数值指标上看一切正常，只有理解内容才能发现。

LLM 擅长判断语义一致性、计划偏差、回答是否答非所问，可以作为规则之外的补充。

但 LLM 不能作为唯一手段：它不稳定、成本高、可能误判。我会用它做「可疑信号」，再结合规则、证据和人工确认。

另外，LLM 检测本身也要有评测集和阈值，不能用一个模型去监控另一个模型却完全不验证。

结论：LLM 适合补充语义异常，规则负责确定性和低成本兜底，两者结合。

3. 怎么强制大模型输出CoT?

要区分「让模型展示推理」和「让模型真的推理」。

第一，Prompt 明确要求分步思考，并给出结构化格式，比如先列已知条件、再分析、再结论。

第二，用独立的 reasoning 字段或 scratchpad，把推理过程和最终答案分开，避免答案污染推理。

第三，给少量示例，展示期望的推理粒度和格式。

第四，对支持推理模式的模型，使用其原生 reasoning 能力；但要注意不要依赖隐藏推理作为唯一依据。

第五，加校验：检查推理是否引用了证据、是否跳步、结论是否与推理一致。

第六，对高风险场景，要求输出假设、证据、反例和置信度，而不只是一段「因为所以」。

注意：强制 CoT 不等于强制正确；模型可能编造推理。所以还要有外部验证和人工兜底。

4. Planner是怎么做任务规划的?

Planner 的核心是把目标变成可执行、可校验的计划。

流程一般是：
1. 理解目标：提取目标、约束、输入资产、成功标准；
2. 识别能力：从 Agent/工具注册表里找可用能力；
3. 分解任务：按依赖拆成步骤，明确每步输入输出；
4. 生成计划：输出结构化 DAG 或步骤列表；
5. 校验：检查依赖闭合、输入输出匹配、权限、预算、是否有环；
6. 执行与重规划：执行中根据结果调整，失败时修复或换路径。

Planner 可以是规则、模板、LLM 或混合。生产环境我倾向于「模板优先 + LLM 处理变化」，因为纯 LLM 规划不稳定、难复现。

关键是计划要可解释：为什么选这个步骤、为什么这个顺序、哪些候选被排除。

5. Planner的输入输出是什么?

输入包括五类：
1. 目标：用户要完成什么，成功标准是什么；
2. 上下文：已知事实、约束、历史状态、输入资产；
3. 能力：可用 Agent/工具的能力标签、Schema、前置条件、成本、权限；
4. 资源：预算、时限、并发限制；
5. 策略：优先固定流程还是允许动态生成，风险等级。

输出是一个结构化计划：
- 步骤列表或 DAG；
- 每步的负责人（Agent/工具）、输入、输出、依赖；
- 验收标准和失败处理；
- 预计成本和时延；
- 决策依据和置信度。

计划必须是机器可校验的，不能只是一段自然语言。否则执行层无法判断依赖是否闭合、数据流是否匹配。

如果信息不足，Planner 应该输出「需要澄清的问题」，而不是硬编一个计划。

6. React是什么?

ReAct 是 Reasoning + Acting 的缩写，核心是让模型交替进行推理和行动。

典型循环是：Thought（分析当前情况、决定下一步）→ Action（选择工具和参数）→ Observation（工具返回）→ 再 Thought，直到完成任务。

它的优点是实现简单、过程可解释，每一步都能看到模型的判断；适合探索性、需要多步工具调用的任务。

缺点是错误会累积，容易陷入重复调用，成本和延迟不可控，而且模型可能编造推理。

工程上要加：最大步数、预算、重复调用检测、工具结果校验、失败重试和人工兜底。

ReAct 不是唯一模式，还有 Plan-and-Execute、Reflection 等；选择取决于任务是否需要边做边调整。

7. 大模型为什么会产生幻觉?

原因是多方面的，不只是「模型不聪明」。

第一，训练目标：模型学的是预测下一个 token，不是事实核查；它会生成「看起来合理」的内容。

第二，知识缺失或过时：训练数据里没有、或者已经变化，模型仍会补全。

第三，上下文问题：检索内容有噪声、冲突或缺失，模型会用自己的先验填补。

第四，指令模糊：问题本身有歧义，模型猜了一个解释。

第五，解码随机性：温度、采样会让模型选择概率较低但更「流畅」的答案。

第六，过度自信：模型没有校准「我不知道」的能力，倾向于给确定答案。

治理要对应到这些原因：RAG 提供依据、Prompt 要求拒答、结构化输出、低温度、外部校验、人工兜底。单靠一句「不要幻觉」没有用。

8. 大模型的温度是怎么影响下一个token的预测概率的?

模型先对词表里的每个 token 输出 logits，再通过 softmax 转成概率。

温度 T 作用在 softmax 前：概率 $\propto$ exp(logit / T)。

T 接近 0 时，分布变得尖锐，最高分 token 的概率接近 1，输出更确定、更保守；T 大于 1 时，分布变平，低概率 token 也有机会被选中，输出更多样、更发散。

Top-P 则是在累积概率达到 P 的候选集合里采样，和温度配合控制随机性。

温度不改变模型「知道什么」，只改变采样分布。低温度不能消除幻觉，只是减少随机发挥；高温度适合创意，不适合事实和工具调用。

工程上通常固定一个参数调另一个，并用评测验证，而不是凭感觉调。

9. 一个Agent系统里面，什么时候应该追问用户，什么时候自己推理?

判断标准是「信息不足的代价」和「猜错的代价」。

应该追问：缺少必填参数、需求有歧义、多个意图无法区分、置信度低、涉及高风险或不可逆操作、用户的约束互相冲突。

可以自己推理：信息足够、歧义小、猜错的代价低且可回滚、可以通过检索/工具补全。自己推理时要明确说明假设，并在结果里标注。

工程上可以设一个「澄清预算」：不能每个问题都问，也不能什么都不问。低风险默认推断，高风险必须确认。

还要结合对话上下文：用户已经说过的信息不要重复问；能通过工具查到的不要问用户。

好的 Agent 不是「什么都问」，也不是「什么都猜」，而是知道哪些信息必须确认。

10. 幻觉的各种治理手段，优缺点分别是什么？行为限制在什么阶段做?

治理手段可以分四类。

数据/检索：RAG、引用、知识库更新。优点是直接提供依据；缺点是对检索质量依赖大，检索错会放大。

Prompt/生成：约束输出、要求引用、允许拒答、结构化输出、低温度。优点是成本低；缺点是不能根除，模型仍可能忽略指令。

验证/后处理：规则校验、事实校验、交叉验证、引用一致性检查。优点是能拦截明显错误；缺点是需要领域规则，覆盖不全。

人工/流程：审批、人工复核、灰度、回滚。优点是可靠；缺点是成本高、影响效率。

行为限制要分阶段：
- 生成前：限定知识来源、权限和工具；
- 生成中：约束格式、要求证据、低温度；
- 生成后：校验、引用检查、敏感信息过滤；
- 执行前：高风险动作审批；
- 上线后：监控、评测、回滚。

没有单一手段能解决幻觉，要按风险分层组合。

11. 如果Agent中间步骤已经跑偏了，怎么发现?

靠「计划对照 + 状态校验 + 外部证据」。

第一，计划对照：每步完成后检查是否还在原计划上，是否完成了预期子目标；偏离超过阈值就重规划。

第二，状态校验：检查必填字段、依赖是否满足、输出是否符合 Schema；不合法就中断。

第三，进度检测：连续几步没有实质进展、重复调用同一工具、Token 消耗异常，判定为停滞。

第四，外部证据：检索结果、工具返回、业务规则与模型结论不一致时，标记可疑。

第五，自检/评审：让模型复述目标、检查结论是否有证据；高风险场景用独立 verifier。

第六，人工兜底：关键节点设审批或抽检。

发现跑偏后不要直接继续，应该回到最后一个可信检查点，重规划或转人工。

12. 怎么降低首Token延迟?

首 Token 延迟（TTFT）主要来自排队、预填充和网络。

第一，流式输出：模型一产生 token 就推给客户端，不要等全部生成完。

第二，Prompt Caching：复用稳定的系统提示和长上下文，减少预填充计算。

第三，缩短输入：裁剪无关上下文、工具结果和检索内容，输入越短预填越快。

第四，模型路由：简单任务用小模型或更快的推理服务；对延迟敏感的场景单独设路由。

第五，连接与部署：连接复用、就近接入、减少网关跳数；必要时用边缘节点。

第六，并发与排队：入口限流、优先级调度，避免请求在队列里等太久。

第七，预热和批处理：对稳定流量预热模型；批处理只适合非交互场景。

要区分 TTFT 和总时长：TTFT 主要影响体验，总时长影响成本和并发。优化时要分别看指标。

13. 支付等高敏感幻觉，HITL怎么做?

支付场景的原则是「模型不能单独决定不可逆动作」。

第一，强制人工审批：Agent 只能生成支付建议或草稿，真正执行必须由人确认。

第二，动作级审批：展示收款方、金额、币种、用途、证据和风险提示，人确认的是这一笔，不是「这个 Agent 有支付权限」。

第三，确定性校验：金额上限、收款方白名单、频率限制、重复支付检测，全部由规则执行，不交给模型。

第四，双人复核：高风险或大额支付需要两个人。

第五，执行前再校验：审批后参数不能变；执行前再检查一次余额、限额和幂等。

第六，审计与回滚：记录谁批的、依据什么；失败或异常时有取消/退款流程。

第七，模型输出只作为参考，不能作为唯一依据。

HITL 不是「加一个确认按钮」，而是把不可逆风险交给可负责的人。

14. 为什么在复杂Agent闭环环境中，仅靠rag无法解决幻觉?

因为 RAG 只解决「知识依据」，但复杂 Agent 的错误来源不止知识。

第一，规划错误：步骤漏了、顺序错了、依赖没闭合，RAG 再准也做不对。

第二，工具错误：参数错、权限不足、工具返回错误，模型可能把错误结果当真。

第三，状态错误：上下文丢了、检查点不一致、重复执行，导致结论基于错误状态。

第四，推理错误：模型把证据推错，或者忽略了反例。

第五，环境变化：检索时和执行时状态不同，答案过期。

第六，多步累积：每一步小误差，最后放大成错误结论。

所以闭环环境需要：计划校验、工具幂等与结果校验、状态管理、证据链、执行后验证、人工兜底。RAG 是其中一层，不是全部。

15. 从数据，检索，生成三个方法，系统降低agent幻觉的具体手段?

数据侧：保证知识质量。去重、清洗、标注来源和时间、版本管理、权限控制；对冲突数据做治理；定期更新和失效旧内容。

检索侧：提高证据质量。混合检索、query 改写、metadata 过滤、rerank、阈值控制、多样性；召回不足时明确说不足，不要硬答；把检索结果和分数记录下来。

生成侧：约束模型。要求基于资料回答、给出引用、资料不足时拒答；结构化输出、低温度、少用自由发挥；关键结论做后校验。

再加两层：
- 执行侧：高风险动作审批、工具结果校验、幂等；
- 运营侧：评测集、监控、人工抽检、失败回流。

核心思路是：让模型「有依据、被约束、可验证、可兜底」，而不是指望它自己不变错。

CodingAgent

1. 如何验证AI生成代码的准确性?

我会做「静态检查 + 动态执行 + 人工评审」三层。

静态：编译/语法检查、类型检查、lint、依赖检查、安全扫描；看是否引入漏洞、死代码、不兼容 API。

动态：在沙箱里跑单元测试、集成测试、回归测试；没有测试就先生成测试，但测试本身也要审查，防止「测试跟着错误代码写」。

差异审查：看 diff 是否最小、是否改了无关文件、是否删除关键逻辑；结合仓库上下文判断。

人工：关键模块、权限/支付/数据迁移等高风险代码必须人工 review。

工程上把 AI 代码当成普通 PR：CI 门禁、代码评审、灰度发布、可回滚。不能因为「模型说对了」就合并。

最后记录生成用的模型、Prompt、上下文和测试结果，方便追溯。

2. CodeRag系统该怎么设计?

代码检索和文档检索不一样，核心是「符号、依赖和上下文」。

索引侧：
1. 按仓库解析 AST，提取文件、类、函数、方法、调用关系、导入关系；
2. 按语义边界切分（函数/类），保留文件路径、包名、签名、注释；
3. 生成 embedding，同时保留关键词和符号索引；
4. 建依赖图，支持「谁调用谁」「改这个会影响谁」。

检索侧：
1. 先识别问题涉及的符号、文件、错误信息；
2. 混合检索：向量 + 关键词 + 符号精确匹配；
3. 沿依赖图扩展上下文，取调用方、被调用方、测试和相关配置；
4. 按 Token 预算组装，优先签名和关键实现，不整文件塞入。

生成侧：让模型基于检索到的代码和接口回答，要求引用文件/行号；改代码时输出 diff，并在沙箱里跑测试。

评测：用真实 issue/PR 做集，看文件召回率、补丁正确率、测试通过率和人工修改率。

关键是「检索到正确的代码上下文」，而不是「把整个仓库塞给模型」。

LLM网关

1. LLM网关怎么设计?

我会把网关定位成「统一入口 + 流量治理 + 成本与可观测」，不让业务直接调用厂商 SDK。

核心模块：
1. 统一 API：把不同厂商的请求/响应/流式格式收敛；
2. Provider 适配：每个厂商一个适配器，只处理协议差异；
3. 路由：按模型能力、成本、延迟、租户、健康度选择；
4. 限流与配额：按用户/租户/模型/全局分层；
5. 重试与熔断：只在一层重试，跨模型 fallback；
6. 缓存：精确缓存、Prompt Caching、语义缓存（按场景）；
7. 成本与用量：Token 统计、账单、预算告警；
8. 可观测：延迟、TTFT、错误率、429、降级次数、trace；
9. 安全：密钥管理、租户隔离、内容 Guardrails、审计；
10. 配置：模型、路由、限流、灰度都可配置、可回滚。

关键设计是 provider 和 gateway 分离：provider 只回答「这个请求怎么写」，gateway 回答「用哪个、挂了怎么办、花多少」。

重试不要叠加，全局预算封顶，避免重试风暴。

MCP

1. MCP是为了解决什么问题提出的?

MCP（Model Context Protocol）主要解决「模型应用和外部能力对接碎片化」的问题。

没有 MCP 时，每个 Agent 框架、每个工具提供方都要做一遍适配，形成 M×N 的集成成本；工具发现、参数格式、鉴权、传输方式各不一样。

MCP 提供了一套标准协议：客户端可以发现服务端有哪些工具、资源和提示词，按统一格式调用，传输层可以复用。这样工具提供方做一次，多个客户端都能用，接近 M+N。

它还把「能力」从应用里解耦出来：工具、资源、提示词可以作为独立服务提供。

但 MCP 不解决可靠性问题，幂等、权限、沙箱、超时、审计仍然要在执行层做；第三方服务端的内容也要按不可信数据处理。

2. 有没有了解替代传统MCP的方法?

有几种思路，但定位不完全一样。

第一，OpenAPI/Function Calling：用标准 API 描述和模型调用意图，生态成熟，但没有统一的工具发现和资源协议，适配仍需自己做。

第二，插件/扩展机制：比如 IDE 插件、Agent 框架的 tool registry，耦合特定平台，复用性有限。

第三，A2A（Agent-to-Agent）类协议：更关注 Agent 之间的协作和任务委派，不是工具接入。

第四，直接 SDK/自定义 RPC：简单场景够用，但跨团队、跨语言复用时成本高。

第五，把工具封装成内部统一网关：所有工具走同一套注册、鉴权、审计，这其实是「自建 MCP」。

MCP 的优势是标准化和生态，劣势是协议层增加了复杂度和信任边界。选择时看是否需要跨客户端复用、是否有第三方工具生态；内部小系统用统一网关可能更简单。

3. SSE是什么?

SSE（Server-Sent Events）是一种基于 HTTP 的服务端单向推送技术。

客户端发起一个普通 HTTP 请求，服务端保持连接，返回 `Content-Type: text/event-stream`，然后持续发送 `data: ...` 格式的事件。客户端用 EventSource 或 fetch stream 读取。

特点：
- 单向：服务端推、客户端收；
- 基于 HTTP，兼容代理和防火墙；

- 支持自动重连和 `Last-Event-ID`；
- 文本协议，实现简单。

在 LLM 场景里，SSE 常用于流式输出 token；在 MCP 里，早期远程传输也用过 SSE。

工程上要注意：事件 seq、心跳、背压、断线重连、客户端断开后取消上游、部分结果状态。

和 WebSocket 比，SSE 更简单、单向；需要双向实时通信时才用 WebSocket。

Benchmark系统

1. LLM-As-Judege是否存在偏差?

存在，而且偏差来源不少。

第一，位置偏差：更偏好排在前面的答案。

第二，长度/格式偏差：更长的、格式更漂亮的答案容易得高分。

第三，自我偏好：模型可能偏爱自己或同家族模型生成的答案。

第四，宽容偏差：倾向于给高分，尤其在评分标准模糊时。

第五，领域偏差：对不熟悉领域判断不准。

第六，提示偏差：评分 Prompt 的措辞、示例会影响结果。

所以 LLM-as-Judge 不能当成绝对真理，要校准、要多种评审、要和人工对齐。高风险场景不能只靠它做决策。

2. LLM-As-Judege怎么消除偏差?

不能完全消除，但可以显著降低。

第一，交换位置：A/B 两个答案随机或对称交换，取一致结果。

第二，明确评分标准：给 rubric，按维度打分，而不是一个笼统的 1-10 分。

第三，多评审：多个模型/多次采样，取平均或投票；分歧大的样本转人工。

第四，盲评：不告诉评审模型答案来自哪个模型。

第五，参考对照：给标准答案或关键要点，减少主观发挥。

第六，控制格式：统一长度、格式，避免风格干扰。

第七，校准：用人工标注样本评估评审模型，调整阈值和 Prompt。

第八，统计显著性：不要用几个样本下结论。

最后，关键决策仍要人工抽检，LLM 评审只是提效工具。

3. 评测指标有哪些?

我会分四层。

结果层：任务成功率、端到端完成率、人工修改率、转人工率、用户采纳率。

质量层：答案正确性、忠实度（是否忠于证据）、引用准确率、幻觉率、拒答准确率、格式合规率。

过程层：意图识别准确率、路由准确率、检索 `Recall@K / NDCG`、工具选择正确率、参数正确率、步骤可执行率。

工程层：`P50/P95` 延迟、`TTFT`、`Token` 成本、重试率、失败恢复率、缓存命中率。

不同任务选不同指标组合，不能只用一个总分。指标要能按版本、场景、租户下钻，并和人工评估对齐。

Agent自进化

1. wikiskill的具体流程?

我理解 `WikiSkill` 的核心是「把执行经验沉淀成可复用的技能文档」，而不是让模型在线改自己。

大致流程：
1. 采集轨迹：记录任务目标、步骤、工具调用、结果、失败原因；
2. 复盘归因：区分成功是偶然还是可复用，失败根因是什么；
3. 技能抽取：提炼触发条件、前置条件、步骤、工具、参数、验收标准和反例；
4. 写入技能库：用统一模板写成 `Wiki/技能文档`，带版本、来源、适用范围；
5. 检索复用：新任务先检索相关技能，作为上下文或计划模板；
6. 验证：离线评测 + 小流量灰度，确认技能真的有效；
7. 反馈迭代：根据使用结果更新、降权或废弃技能。

关键是「受控沉淀」：技能要可追溯、可验证、可回滚，不能让一次偶然成功直接变成全局规则。

动态workflow

1. 基于能力路由的动态workflow和传统workflow比有什么区别?

传统 Workflow 的步骤和顺序是人预先定义的，确定性强、可复现、容易测试，但面对变化不灵活。

基于能力路由的动态 Workflow 在运行时根据任务目标、输入资产和 Agent 能力选择节点、生成依赖图。它更灵活，能处理没有模板的新任务，也能根据中间结果调整路径。

区别主要在四点：
1. 控制流来源：人定义 vs 运行时决策；
2. 可复现性：传统更稳，动态需要版本、快照和轨迹才能复现；
3. 成本与延迟：动态多了规划和校验开销；
4. 失败处理：动态需要更强的校验、预算和回退机制。

生产上我通常混合：主干用固定/模板化 Workflow，动态生成只在没有合适模板时兜底，并且必须经过确定性校验才能执行。

不是「动态一定更好」，而是「在变化大、模板覆盖不了的场景才值得」。

排障agent

1. 排障agent有哪些组件?

我会拆成八个组件。

1. 故障接入：告警、日志、指标、变更记录、执行轨迹，统一成故障上下文。
2. 分类器：按症状、影响面、组件做分层分类，先缩小范围。
3. 假设生成：生成多个根因假设，每个假设对应可验证的检查动作。
4. 诊断规划：根据信息增益选择下一步查什么，动态组织诊断 `Workflow`。
5. 工具网关：只读查询、权限控制、超时、审计；恢复类工具单独授权。
6. 证据管理：记录每条证据的来源、时间、置信度，支持结论追溯。
7. 恢复控制：按风险分级执行恢复，高风险审批，执行前验证、执行后可回滚。
8. 经验库：结构化保存已确认的故障案例，支持检索复用和反馈更新。

外围还需要 `Guardrails`、预算控制、人工接管和可观测性。核心是「证据驱动 + 受控恢复」，不是让模型自由发挥。

业务场景识别与价值评估

1. 如果让你负责公司 AI 应用建设，你会如何选择第一个落地场景？

我会按「高频、痛点明确、数据可得、结果可验证、风险可控、有人买单」六个标准选。

第一，高频：每天/每周都发生，收益能累积，而不是一次性需求。
第二，痛点明确：有明确的等待、重复劳动或错误成本，不是「看起来可以试试」。
第三，数据可得：有文档、日志、历史案例或业务系统接口，不用从零造数据。
第四，结果可验证：对错能判断，最好有客观指标或专家能快速审核。
第五，风险可控：失败可回滚，不涉及资金、权限、合规等高风险动作，或者能用人工兜底。
第六，有人买单：业务方愿意参与、愿意用，最好有一个支持你的 champion。

第一个场景我会选「高频 + 低风险 + 可验证」的窄场景，先做端到端闭环，用真实数据证明价值，再扩展。不会一上来做全流程自动化。

2. 如何通过业务访谈发现 AI 应用机会？

我会按「流程–痛点–数据–价值」四步做。

第一，画流程：跟业务人员走一遍真实工作流，从输入到输出，标出每个环节的耗时、系统、角色和交接点。

第二，找痛点：问具体问题，比如「哪一步最耗时」「哪里最容易出错」「哪些工作是重复的」「如果多一个人你最想让他做什么」。不要问「你觉得 AI 能帮你什么」，用户很难回答。

第三，看数据：这个环节有没有文档、日志、历史记录、系统接口；数据质量如何；能不能拿到。

第四，算价值：节省多少时间、减少多少错误、提升多少吞吐；谁受益、谁买单、谁会被影响。

还要观察「影子流程」：员工私下用 Excel、脚本、聊天工具补的环节，往往是最真实的痛点。

最后按「价值 × 可行性 × 风险」排序，选一个窄场景做试点，用结果说话。